CityTech TTP Summer Bootcamp

Ionic and Git Branching

2026-06-30

Agenda

1. Review
2. Follow-Ups: Ionic CLI
3. Pages and Components
4. Routing and Props
5. Handling Input: React Hook Form
6. State Variables
7. Git Branches

Review

What are the most important things you remember from yesterday?

Follow Up: Installing the Ionic CLI

Yesterday `yarn dlx @ionic/cli` failed with a missing **`minimist`** package. Installing the CLI **globally** fixed it.

```
npm install -g @ionic/cli   # install once, globally
ionic start myApp tabs --type=react
```

- Run **`ionic`** directly, not through `yarn dlx`
- Using **npm** for this one global tool is fine, even in a yarn project

Why `yarn dlx` Broke

`yarn dlx` runs in a throwaway **Yarn PnP** environment with **strict** module resolution.

- Under **PnP**, a package can only import dependencies it **explicitly declares**
- The Ionic CLI uses **`minimist`** without declaring it, so PnP can't find it
- A global **npm** install uses a hoisted `node_modules`, where that loose import resolves, so it works

Follow Up: Finish Your Scattergories Repo

Before you move on, make sure your **Scattergories** project repo includes:

- A SQL dump named `dump.sql` so the database can be recreated
- A `README` covering the bare minimum:
  - How to **get the project running** (install, set up the DB, start it)
  - How to **play** the game

A new person should be able to clone the repo and get it working from the README alone.

Pages

A **page** is one full-screen view in your app, usually one per route.

In Ionic, a page is a component that renders an `<IonPage>`. Pages are what the router moves between, and they give you a header, scrolling content, and native transitions without extra work.

Example Page Component

```
const Home = () => (
  <IonPage>
    <IonHeader>
      <IonToolbar><IonTitle>Home</IonTitle></IonToolbar>
    </IonHeader>
    <IonContent>Welcome!</IonContent>
  </IonPage>
);
```

Components

A **component** is a reusable piece of UI. Write it once, then reuse it across pages. Components keep your screens organized and save you from repeating the same markup. Components can accept **props** to provide options or settings on how or what they should render, increasing their usability and flexibility.

Example Component

```
// a component you write yourself
const Greeting = ({ name }: { name: string }) => {
  return <p>Hello, {name}!</p>;
};

// reuse it on any page
<Greeting name="Ada" />
```

Pages vs Components

- A **page** is a whole **screen** (one per route); a **component** is a **piece of UI** inside a page
- Pages live in `src/pages/`; components in `src/components/`

Technically, a **page is a component** too: the difference is how you use it, not what it is.

Rule of thumb: if the router navigates to it, it's a **page**; if you reuse it inside screens, it's a **component**.

The Ionic Router

The **router** decides which page to show for a given URL, and handles moving between pages (with the back button and native transitions).

By default, the Ionic starter sets it up for you in `src/App.tsx`.

Example: Router Setup

```tsx
// src/App.tsx
import { IonApp, IonRouterOutlet } from "@ionic/react";
import { IonReactRouter } from "@ionic/react-router";
import { Route } from "react-router-dom";

const App = () => (
  <IonApp>
    <IonReactRouter>
      <IonRouterOutlet>
        <Route path="/home" component={Home} />
        <Route path="/about" component={About} />
      </IonRouterOutlet>
    </IonReactRouter>
  </IonApp>
);
```

Each `<Route>` maps a path to a page; navigate with `routerLink="/about"`.

Aside: Router Layouts

Ionic wraps the router in different **navigation shells**, which is what the starter templates give you:

- **Tabs**: a bottom tab bar, where each tab keeps its own page stack (`tabs` starter)
- **Side menu**: a slide-out drawer of links (`sidemenu` starter)
- **Plain**: just pages and routes, navigating forward and back (`blank` starter)

The routing idea is the same; only the **shell** around it changes.

URL Parameters

A route can capture part of the URL as a **parameter**, written with a colon like `:id`.

That lets one page handle many records: `/items/1`, `/items/2`, and so on, instead of a separate route for each. The page reads the value to know what to show.

Example: URL Parameters

Declare the param in the route, then read it in the page with `useParams`.

```
// in src/App.tsx
<Route path="/items/:id" component={ItemDetail} />
```

```
// inside the ItemDetail page component
import { useParams } from "react-router-dom";

const { id } = useParams<{ id: string }>();   // "1", "2", ...
```

Routes: Ionic vs Express

Same `:id` path syntax, on the server and in the client.

```
// Express (server)
app.get("/about", handler);
app.get("/items/:id", handler); // :id -> req.params.id
```

```
// Ionic (client)
<Route path="/about" component={About} />
<Route path="/items/:id" component={ItemDetail} /> // :id -> useParams()
```

Props

Props are the inputs you pass into a component, like arguments to a function or attributes on an HTML tag.

The parent passes values down, and the component uses them to decide what and how to render. Same component, different props, different result.

Example: IonButton Props

Same component, different props:

```
<IonButton>Default</IonButton>
<IonButton color="danger">Delete</IonButton>
<IonButton fill="outline">Outline</IonButton>
<IonButton expand="block">Full width</IonButton>
<IonButton disabled>Disabled</IonButton>
```

Each prop changes how the button looks or behaves.

Code Together

Let's play with Ionic **pages** and **components** together. No spec, just explore.

Handling Input

In our **React chat app** (`src/App.tsx`), forms were handled the **vanilla** way:

```
<form onSubmit={(event) => {
  event.preventDefault();
  const text = event.target.incoming_text.value;
  document.getElementById('incoming_text').value = '';
}}>
```

It reads and clears the input straight from the DOM. The code even notes this "is not best practice but works for now." The cleaner pattern is **controlled inputs** that keep the value in state.

Code: `demos/react-chat-app`

React Hook Form

React Hook Form is a popular library for building forms in React. You register your inputs with a hook, and it handles the values, validation, and submission.

Benefits:

- **Less boilerplate**: no `useState` and `onChange` for every field
- **Built-in validation** with easy error messages
- **Fast**: it minimizes re-renders as you type
- Small, with strong **TypeScript** support

Docs: https://react-hook-form.com

Example: React Hook Form

```
import { useForm, Controller } from "react-hook-form";
import { IonInput, IonButton } from "@ionic/react";

const NewMessage = () => {
  const { control, handleSubmit } = useForm();

  const onSubmit = (data) => console.log(data.text);

  return (
    <form onSubmit={handleSubmit(onSubmit)}>
      <Controller
        name="text"
        control={control}
        render={({ field }) => (
          <IonInput
            value={field.value}
            onIonChange={(e) => field.onChange(e.detail.value)}
          />
        )}
      />
      <IonButton type="submit">Send</IonButton>
    </form>
  );
};
```

State Variables

A **state variable** is a value a component remembers between renders. When you change it, React **re-renders** the component to show the new value.

You create one with the `useState` hook: it gives you the current value and a setter.

```
const [count, setCount] = useState(0);
setCount(count + 1); // updates the value and re-renders
```

We have seen this already (the chat app kept its messages in `useState`); same idea.

Array Destructuring in `useState`

`useState` returns an **array**: the current value, then a setter function.

```
const result = useState(0);
const count = result[0];     // current value
const setCount = result[1];  // setter
```

Array destructuring unpacks that array in one line, and you choose the names:

```
const [count, setCount] = useState(0);
```

Same idea as `const [a, b] = [1, 2]`. By convention: `[thing, setThing]`.

Code Together: React Hook Form

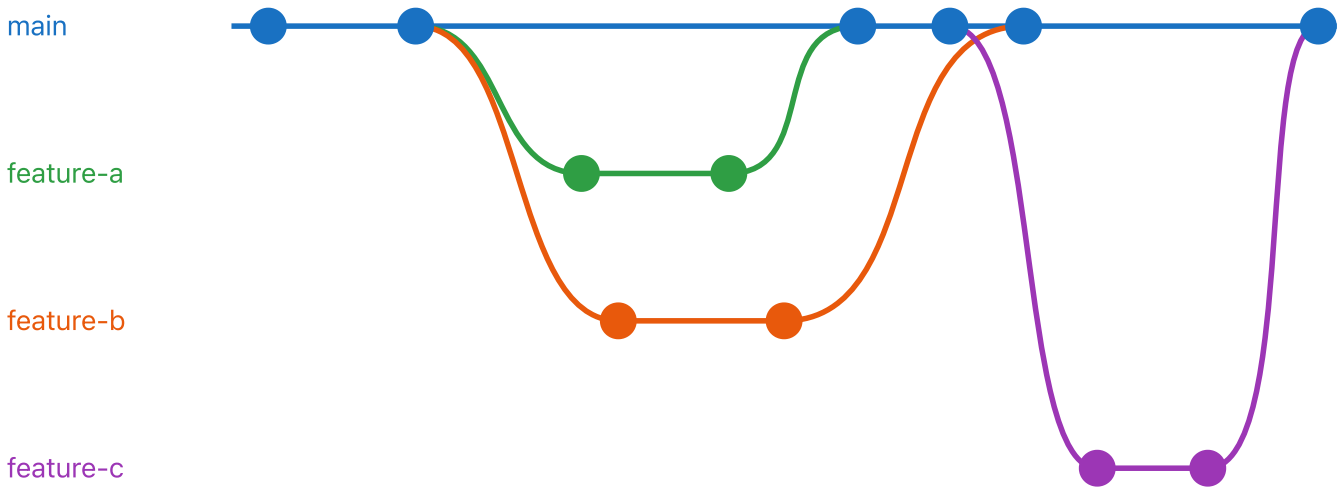
Let's build a small form with **React Hook Form** together.

Git Branches

A **branch** is a separate line of work in your repo. You can commit on a branch without touching `main`, then **merge** it back when it is ready.

- Try a feature or fix in **isolation**
- Let teammates work in **parallel** without stepping on each other
- Keep `main` **stable** while work is in progress

Branching and Merging



`feature-a` and `feature-b` split off the **same** commit but **merge** back at different points; `feature-c` branches off later.

Branch Commands

```
git branch                # list your branches
git checkout -b feature   # create a new branch and switch to it
git push origin feature   # push the branch to GitHub
git pull origin feature   # pull the branch from GitHub
```

Push and Pull the Current Branch

Typing `origin <branch>` every time gets old. You can configure git to **push and pull the current branch** by default.

```
# push: create/use a matching branch on origin automatically
git config --global push.default current
git config --global push.autoSetupRemote true
```

After this, from any branch you can just run:

```
git push   # pushes the current branch to its matching remote branch
git pull   # pulls the current branch
```

Once a branch is **tracking** its remote (the first push sets this up), bare `git pull` knows what to fetch.

Branch Naming Conventions

A team usually agrees on a naming style so branches stay readable. Common ones:

- **Type prefix**: `feature/`, `bugfix/`, `hotfix/`, `chore/` — e.g. `feature/login-form`
- **Ticket number**: tie the branch to an issue, e.g. `JIRA-35-login-form` or `123-fix-signup`
- **Author prefix**: `jon/login-form` so it's clear who owns it
- **Plain description**: just `login-form` for small projects

Conventions:

- Use **lowercase** and **hyphens**, no spaces
- Keep it **short but descriptive**
- Pick one style as a team and **stay consistent**

Why Branches?

- **Isolation**: develop a feature or fix on its own branch; if it breaks or gets abandoned, `main` is untouched
- **Parallel work**: teammates each work on their own branch at the same time, without blocking each other
- **A stable `main`**: `main` always holds known-good, deployable code; messy work-in-progress stays off it
- **Code review / PRs**: a branch is the unit you open a pull request from, so others can review before it merges
- **Cheap and fast**: a branch is just a pointer to a commit, so create, switch, and delete them freely

Practice by Playing

Some games for practicing git:

1. Oh My Git! — https://ohmygit.org/
2. Game4Git — https://game4git.games/
3. Git Game — https://github.com/git-game/git-game
4. Learn Git Branching — https://learngitbranching.js.org/

Today's Exit Ticket

- [ ] Explain **Ionic pages and components**
- [ ] **CRUD** pages in `IonRouter`
- [ ] Use **Ionic components** with different **props**
- [ ] Explain and use **React Hook Form**
- [ ] Explain and use **state variables**
- [ ] Explain and **CRUD git branches**